Міністерство освіти і науки України
Державний університет „Житомирська політехніка”

Кафедра КТуМтаТ
Група: ТР-25-1

Теорія ймовірностей і математична статистика

Практична робота №5-6

«Умовна ймовірність, теореми додавання та множення ймовірностей у діагностиці та аналізі програмних систем»

Перевірив: Лисенко М. С.

Виконав: Наумчук А. В.

					ІКТР.420001.123-ЗЛ		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Наумчук А. В.			Звіт з практичної роботи	Літ.	Арк. 1 / Аркушів
Перевір.		Лисенко М. С.					
Керівник						ФІКТ, гр. ТР-25-1	
Н. контр.							
Затверд.							

Мета: засвоїти практичні навички застосування теорем додавання та множення ймовірностей для сумісних, несумісних, залежних та незалежних подій; навчитися обчислювати умовні ймовірності, будувати програмні моделі оцінки надійності складних розподілених систем (серійно-паралельних схем) та здійснювати діагностику збоїв ПЗ методом Монте-Карло.

Короткі теоретичні відомості:

БЛОК 1.

Умовна ймовірність та незалежність подій Теоретичною базою блоку є поняття умовної ймовірності P(A | B), що визначає ймовірність події A за умови настання події B. Для залежних подій ймовірність їх спільного настання обчислюється за теоремою множення: P (A ∩ B) = P(B) * P(A | B). Дві події визнаються незалежними тоді й лише тоді, коли виконується рівність P (A ∩ B) = P(A) * P(B).

БЛОК 2 . (Варіант 1)

Робота двох маршрутизаторів розглядається як незалежні події. Тому ймовірність відмови обох пристроїв одночасно знаходиться за теоремою множення для незалежних подій: P (A ∩ B) = P(A) * P(B). Оскільки маршрутизатори можуть відмовити як окремо, так і разом, ці події є сумісними, тому ймовірність відмови хоча б одного з них обчислюється за теоремою додавання: P (A ∩ B) = P(A) + P(B) - P (A ∩ B).

БЛОК 3.

Складна IT-інфраструктура розраховується за двома правилами:

- **Послідовне з'єднання (серійне):** Відмова хоча б одного елемента призводить до відмови всього ланцюга. Надійність дорівнює добутку безвідмовної роботи всіх вузлів: P = $p_1 * p_2 ... * p_n$

✡ **Паралельне з'єднання (резервування):** Система відмовляє лише тоді, коли відмовляють усі паралельні елементи. Її надійність обчислюється через імовірність протилежної події: P = 1 - (1 -p_1)(1 - p_2).

БЛОК 4. (Варіант 4)

Загальна успішна робота системи складається з двох несумісних подій: штатної роботи основної бази даних (p_1)АБО ситуації, коли основна база відмовляє ($1 - p_1$), але при цьому успішно спрацьовує перемикач (p_s) та підключається резервна база (p_1). Математично це записується як :

$$P = p_1 + (1 - p_1) * p_s * p_2$$

Виконання роботи:

БЛОК 1 .

```
for n in N:
    count_B, count_A, count_A_and_B = 0, 0, 0 # Обнулення лічильників для нової вибірки

    for i in range(n):
        is_B = random.random() < pB
        if is_B:
            count_B += 1    # Фіксуємо настання B
            is_A = random.random() < pA_given_B
        else:
            is_A = random.random() < pA_given_not_B

        if is_A:
            count_A += 1    # Фіксуємо настання A
            if is_B:
                count_A_and_B += 1

    w_B_given_A = count_A_and_B / count_A if count_A > 0 else 0
    frequencies.append(w_B_given_A)
    error = abs(w_B_given_A - pB_given_A_theor)
```

Рисунок 1.1 – Частина коду яка моделює кожне випробовування

```
C:\Users\asusp\AppData\Loca
---------- Блок 1: Симуляція умовної ймовірності ----------
N          | P(A|B) (стат)   | P(A|B) (теор)   | Похибка
-----------------------------------------------------------
100        | 0.72414         | 0.75000         | 0.02586
1000       | 0.76866         | 0.75000         | 0.01866
10000      | 0.76056         | 0.75000         | 0.01056
100000     | 0.75281         | 0.75000         | 0.00281
1000000    | 0.74914         | 0.75000         | 0.00086
```

Рисунок 1.2. – Вихідні дані в вигляді таблиці.

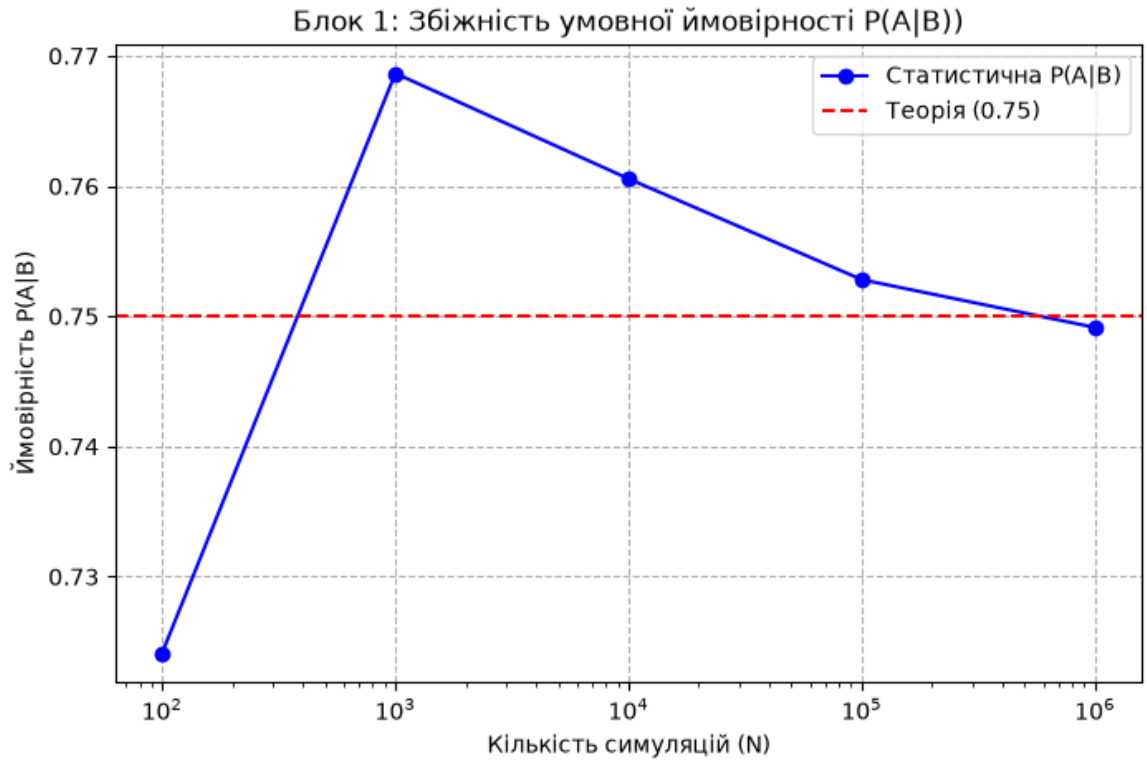


Рисунок 1.3. – Вихідні дані в вигляді графіку. Використана бібліотека matplotlib

БЛОК 2 (Варіант 1)

```
frequencies = []#список для розрахованої частоти

print(f"{'N':<10} | {'P(A U B) (стат)':<15} | {'P(A U B) (теор)':<15} | {'Похибка':<15}")
print("-" * 63)

for n in N:
    count_union = 0
    for i in range(n):
        is_A = random.random() < p_Aheor
        is_B = random.random() < p_Bheor
        if is_A or is_B:# перевірка умови
            count_union += 1# лічильник

    w_union = count_union / n# обчислення статистичної частоти
    frequencies.append(w_union)
    error = abs(w_union - p_AUB)#повертає абсолютне число
```

Рисунок 2.1 - Частина коду яка моделює каскадну відмову маршутизаторів.

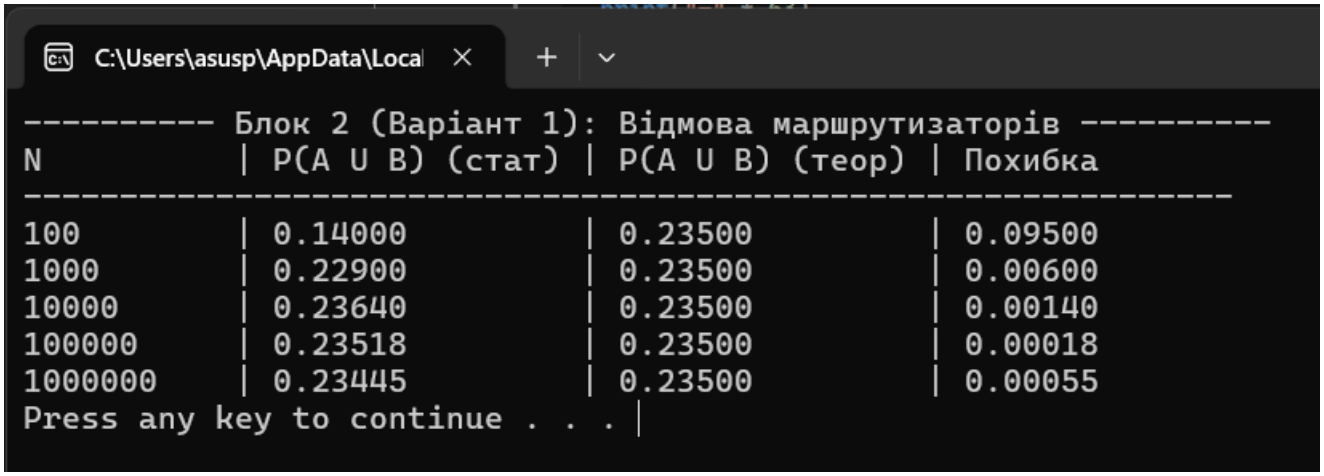


Рисунок 2.2 – Вихідні дані у вигляді таблиці

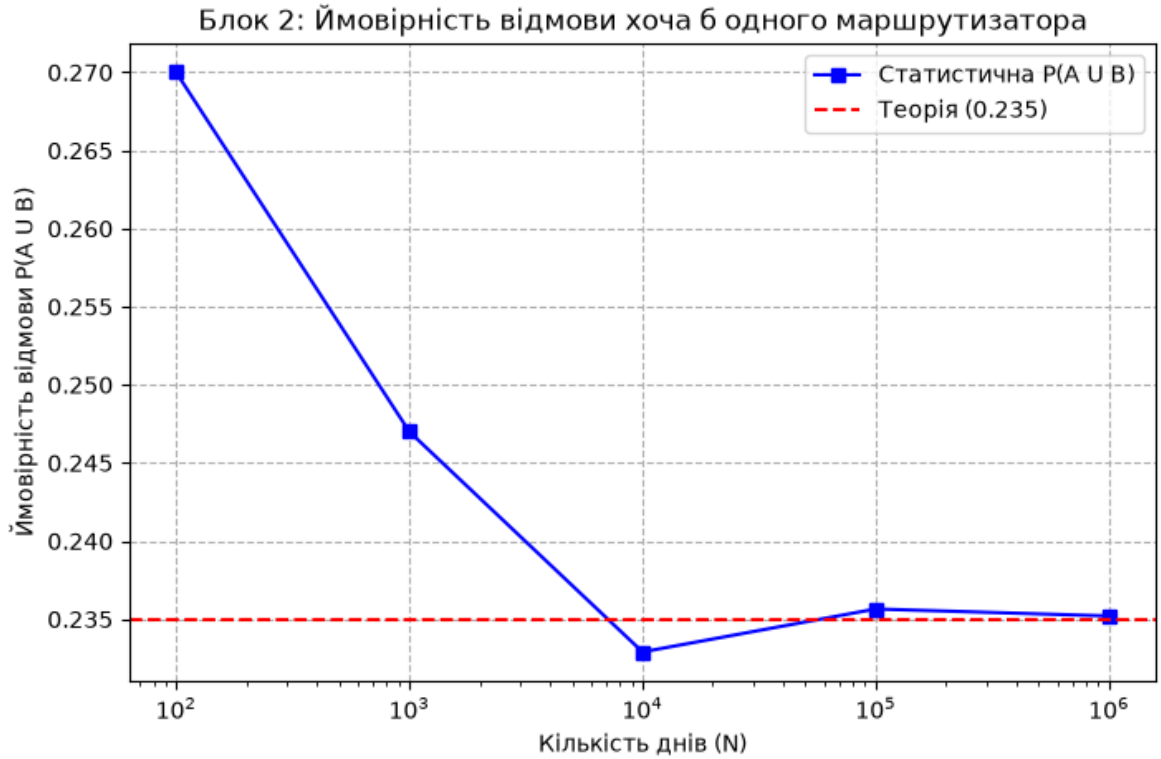


Рисунок 2.3 – Вихідні дані у вигляді графіку

БЛОК 3

```python
for n in N:
    success_count = 0
    for i in range(n):# симуляція кожного окремого компонента
        lb = random.random() < p1_lb# балансувальник навантаження
        web1 = random.random() < p2_web1# перший веб-сервер
        web2 = random.random() < p3_web2# другий веб-сервер
        db1 = random.random() < p4_db1# перша база даних
        db2 = random.random() < p5_db2# друга база даних

        if lb and (web1 or web2) and (db1 or db2):
            success_count += 1

    p_sys = success_count / n
    frequencies_sys.append(p_sys)
    error = abs(p_sys - p_sys_theor)
```

Рисунок 3.1 - Частина коду яка моделює роботу розподіленої системи

```
C:\Users\asusp\AppData\Loca
---------- Блок 2 (Варіант 1): Відмова маршрутизаторів ----------
N          | P(A U B) (стат) | P(A U B) (теор) | Похибка
-----------------------------------------------------------------
100        | 0.19000         | 0.23500         | 0.04500
1000       | 0.20800         | 0.23500         | 0.02700
10000      | 0.23620         | 0.23500         | 0.00120
100000     | 0.23387         | 0.23500         | 0.00113
1000000    | 0.23402         | 0.23500         | 0.00098
```

Рисунок 3.2 – Вихідні дані у вигляді таблиці

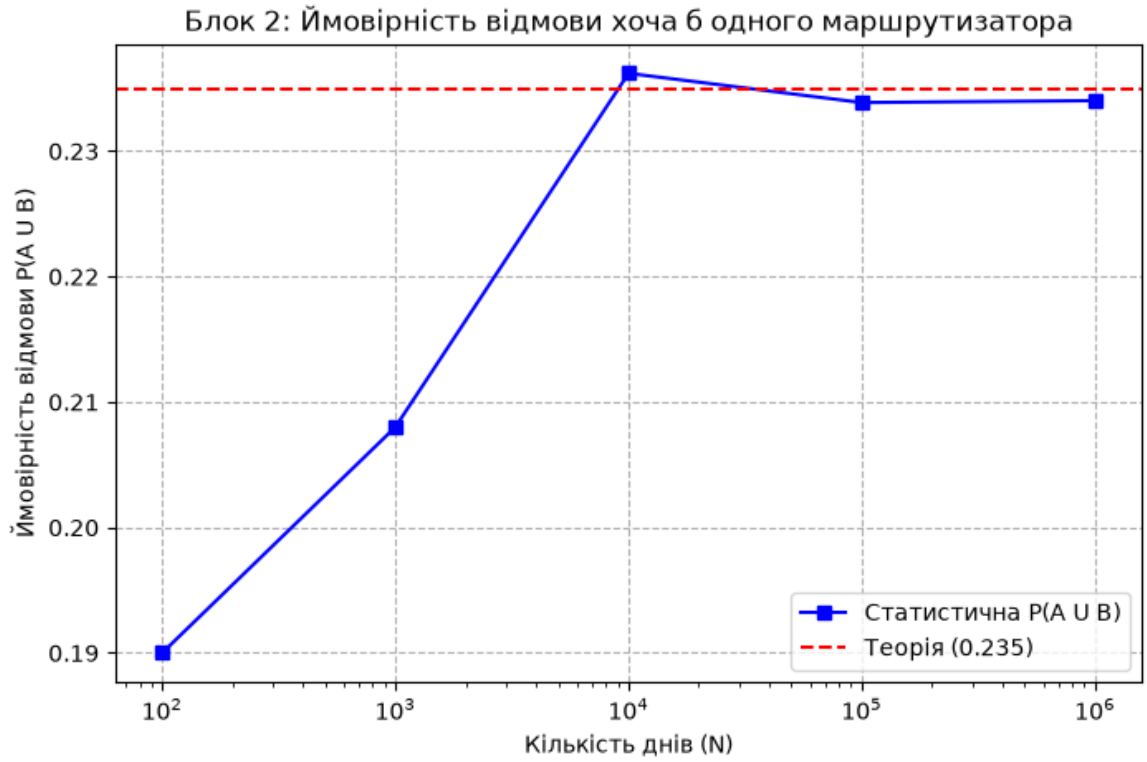


Рисунок 3.3 – Вихідні дані у вигляді графіку

БЛОК 4 (Варіант 4)

```
for n in N:
    success_count = 0
    for i in range(n):#перевірка основної бази даних
        if random.random() < p1:
            success_count += 1#фіксуємо успішну роботу системи
        else:
            if random.random() < ps and random.random() < p2:
                success_count += 1# фіксуємо якщо стався збій

    w_stat = success_count / n
    frequencies.append(w_stat)
    error = abs(w_stat - p_theor)
```

Рисунок 4.1 – Частина коду яка моделює роботу системи резервування у кластері без даних.

```
C:\Users\asusp\AppData\Local
---------- Блок 4 (Варіант 4): Перевірка Failover у кластері БД ----------
N          | P_failover (стат)  | P (теор)    | Похибка
------------------------------------------------------------
100        | 1.00000            | 0.99050     | 0.00950
1000       | 0.99100            | 0.99050     | 0.00050
10000      | 0.99150            | 0.99050     | 0.00100
100000     | 0.99034            | 0.99050     | 0.00016
1000000    | 0.99051            | 0.99050     | 0.00001
```

Рисунок 4.2 – Вихідні данні у вигляді таблиці

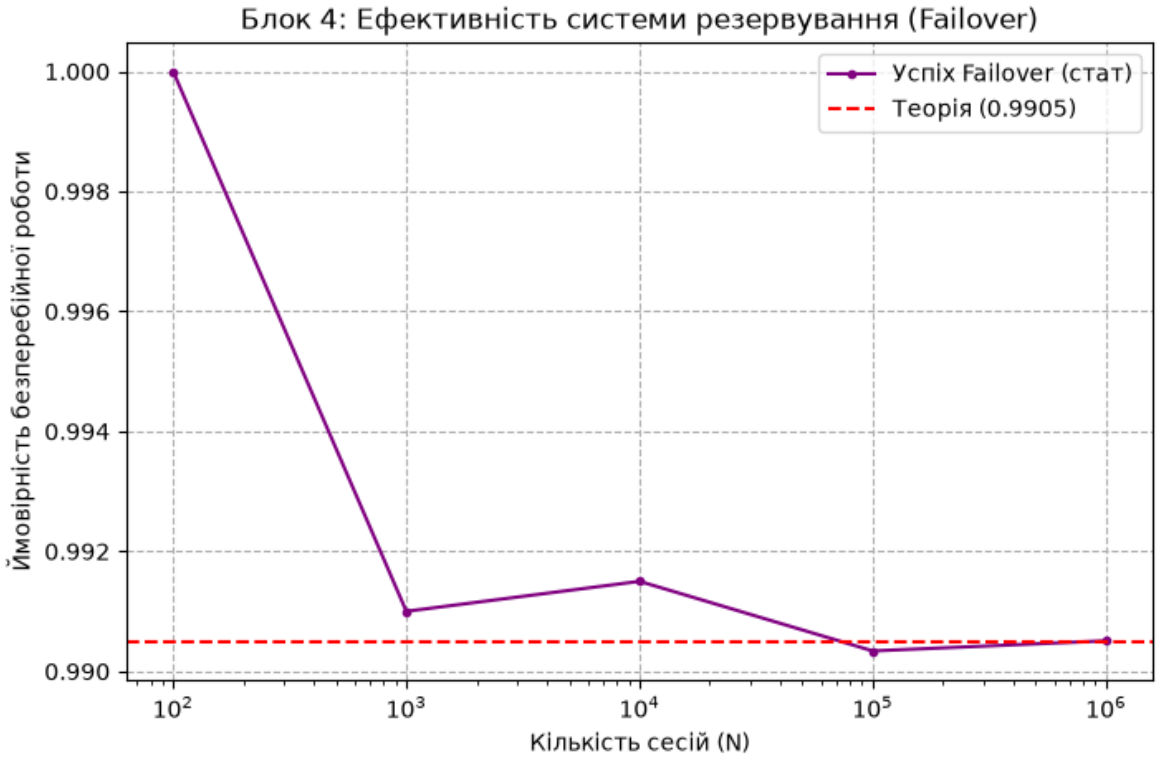


Рисунок 4.3 – Вихідні дані у вигляді графіку

Посилання на репрозиторій:

https://github.com/8sepp8/PR5-6_TheoryOfProbaility_Naumchuk_TR-25-1

ВИСНОВОК

Під час виконання практичної роботи я успішно застосував теореми додавання та множення ймовірностей для розрахунку надійності ІТ-систем. За допомогою методу Монте-Карло я програмно змоделював залежні та незалежні події (каскадні відмови маршрутизаторів, роботу серійно-паралельних вузлів та механізми резервування Failover). Написані скрипти наочно підтвердили дію Закону великих чисел: статистичні частоти на великих вибірках практично ідеально збіглися з точними теоретичними моделями відмовостійкості.